Щучкин Максим, Б07-202

Отчет по лабораторной работе #5

«Вычисление трехмерного интеграла»

Цель: реализовать квадратуру средних прямоугольников для трехмерного случая и агрегировать частичные суммы с помощью коллективной операции MPI_Reduce. (Посчитать трехмерный интеграл).

Текст программы:

```
#include <stdio.h>
#include <stdlib.h>
#include <math.h>
#include <mpi.h>

/* Подынтегральная функция f(x,y,z) = x^2 + y^2 + z^2 */
double f(double x, double y, double z) {
    return x * x + y * y + z * z;
}

/* Автоматическое определение сбалансированной 3D-решетки процессоров */
void get_grid_3d(int np, int *p1, int *p2, int *p3) {
    *p1 = *p2 = *p3 = 1;
    int temp = np;
    for (int i = 2; i * i <= temp; i++) {
        while (temp % i == 0) {
            if (*p1 <= *p2 && *p1 <= *p3) (*p1) *= i;
            else if (*p2 <= *p1 && *p2 <= *p3) (*p2) *= i;
            else (*p3) *= i;
            temp /= i;
        }
    }
    if (temp > 1) {
        if (*p1 <= *p2 && *p1 <= *p3) (*p1) *= temp;
        else if (*p2 <= *p1 && *p2 <= *p3) (*p2) *= temp;
        else (*p3) *= temp;
    }
}

int main(int argc, char *argv[]) {
    int np, mp;
    MPI_Init(&argc, &argv);
    MPI_Comm_size(MPI_COMM_WORLD, &np);
    MPI_Comm_rank(MPI_COMM_WORLD, &mp);

    /* Параметры интегрирования: куб [0,1]x[0,1]x[0,1], сетка 1000^3 */
    double a = 0.0, b = 1.0, c = 0.0, d = 1.0, e = 0.0, g = 1.0;
    int Nx = 1000, Ny = 1000, Nz = 1000;
```

```
    /* Определяем размеры решетки процессоров */
    int p1, p2, p3;
    get_grid_3d(np, &p1, &p2, &p3);

    /* Логические координаты текущего процесса в 3D-решетке */
    int m1 = mp / (p2 * p3);
    int m2 = (mp / p3) % p2;
    int m3 = mp % p3;

    /* Границы подсетки для текущего процесса */
    int i_start = m1 * (Nx / p1);
    int i_end   = (m1 == p1 - 1) ? Nx : i_start + Nx / p1;
    int j_start = m2 * (Ny / p2);
    int j_end   = (m2 == p2 - 1) ? Ny : j_start + Ny / p2;
    int k_start = m3 * (Nz / p3);
    int k_end   = (m3 == p3 - 1) ? Nz : k_start + Nz / p3;

    double hx = (b - a) / Nx;
    double hy = (d - c) / Ny;
    double hz = (g - e) / Nz;
    double hvol = hx * hy * hz;

    /* Локальное суммирование */
    double local_sum = 0.0;
    double t1 = MPI_Wtime();
    for (int k = k_start; k < k_end; k++) {
        double z = e + (k + 0.5) * hz;
        for (int j = j_start; j < j_end; j++) {
            double y = c + (j + 0.5) * hy;
            for (int i = i_start; i < i_end; i++) {
                double x = a + (i + 0.5) * hx;
                local_sum += f(x, y, z);
            }
        }
    }
    local_sum *= hvol;
    t1 = MPI_Wtime() - t1;

    /* Коллективное суммирование на процессе 0 */
    double total_sum = 0.0;
    double t2 = MPI_Wtime();
    MPI_Reduce(&local_sum, &total_sum, 1, MPI_DOUBLE, MPI_SUM, 0,
MPI_COMM_WORLD);
    t2 = MPI_Wtime() - t2;

    fprintf(stderr, "mp=%d t1=%10lf t2=%10lf t3=%10lf int=%22.15le\n",
            mp, t1, t2, t1 + t2, (mp == 0) ? total_sum : local_sum);
```

```
    MPI_Finalize();
    return 0;
}
```

Output:

```
fnbik09@sr210:~/mpc/task5$ mpicc -O2 solution_05.c -o solution_05.px -lm
fnbik09@sr210:~/mpc/task5$ mpirun -np 1 solution_05.px 2>&1 | grep -v Authorization
mp=0 t1=  1.964130 t2=  0.000032 t3=  1.964163 int= 9.999997500002072e-01
fnbik09@sr210:~/mpc/task5$ mpirun -np 8 solution_05.px 2>&1 | grep -v Authorization
mp=7 t1=  0.315379 t2=  0.000030 t3=  0.315408 int= 2.187499687499898e-01
mp=5 t1=  0.316966 t2=  0.000019 t3=  0.316985 int= 1.562499687500172e-01
mp=6 t1=  0.318079 t2=  0.000033 t3=  0.318112 int= 1.562499687500908e-01
mp=3 t1=  0.319702 t2=  0.000039 t3=  0.319741 int= 1.562499687500224e-01
mp=1 t1=  0.320745 t2=  0.000036 t3=  0.320782 int= 9.374996875000997e-02
mp=0 t1=  0.291505 t2=  0.049958 t3=  0.341463 int= 9.999997500002117e-01
mp=2 t1=  0.337785 t2=  0.003556 t3=  0.341341 int= 9.374996875003420e-02
mp=4 t1=  0.341335 t2=  0.000036 t3=  0.341371 int= 9.374996875003315e-02
fnbik09@sr210:~/mpc/task5$ mpirun -np 27 solution_05.px 2>&1 | grep -v Authorization
mp=24 t1=  0.086183 t2=  0.000027 t3=  0.086210 int= 5.361716337566195e-02
mp=17 t1=  0.110838 t2=  0.000026 t3=  0.110864 int= 6.185580534281963e-02
mp=21 t1=  0.113154 t2=  0.000025 t3=  0.113180 int= 3.699578106162725e-02
mp=16 t1=  0.113667 t2=  0.000025 t3=  0.113692 int= 4.520975643605304e-02
mp=13 t1=  0.116603 t2=  0.000033 t3=  0.116636 int= 2.866282998675649e-02
mp=20 t1=  0.117566 t2=  0.000023 t3=  0.117589 int= 5.361716337567185e-02
mp=12 t1=  0.109141 t2=  0.008627 t3=  0.117768 int= 2.047344735295407e-02
mp=22 t1=  0.117570 t2=  0.000020 t3=  0.117589 int= 4.520975643605320e-02
mp=14 t1=  0.109093 t2=  0.008757 t3=  0.117849 int= 4.520975643605191e-02
mp=19 t1=  0.121025 t2=  0.000026 t3=  0.121051 int= 3.699578106162862e-02
mp=11 t1=  0.110700 t2=  0.010023 t3=  0.120723 int= 3.699578106162694e-02
mp=23 t1=  0.120543 t2=  0.000025 t3=  0.120568 int= 6.185580534281963e-02
mp=25 t1=  0.160882 t2=  0.000019 t3=  0.160902 int= 6.185580534283494e-02
mp=9 t1=  0.117096 t2=  0.043931 t3=  0.161027 int= 1.228406471916905e-02
mp=5 t1=  0.119152 t2=  0.042132 t3=  0.161284 int= 3.699578106162625e-02
mp=15 t1=  0.162896 t2=  0.000026 t3=  0.162923 int= 3.699578106162707e-02
mp=7 t1=  0.117055 t2=  0.045507 t3=  0.162561 int= 3.699578106162771e-02
mp=3 t1=  0.117087 t2=  0.045428 t3=  0.162516 int= 1.228406471916878e-02
mp=1 t1=  0.119002 t2=  0.043649 t3=  0.162651 int= 1.228406471916992e-02
mp=18 t1=  0.204704 t2=  0.000032 t3=  0.204737 int= 2.878180568720133e-02
mp=26 t1=  0.206806 t2=  0.000033 t3=  0.206839 int= 7.860142281649152e-02
mp=10 t1=  0.202130 t2=  0.004853 t3=  0.206983 int= 2.047344735296217e-02
mp=6 t1=  0.206560 t2=  0.000381 t3=  0.206941 int= 2.878180568720073e-02
mp=8 t1=  0.209371 t2=  0.000065 t3=  0.209436 int= 5.361716337567165e-02
mp=0 t1=  0.205485 t2=  0.004048 t3=  0.209533 int= 9.999997500000132e-01
mp=2 t1=  0.110190 t2=  0.099696 t3=  0.209886 int= 2.878180568719206e-02
mp=4 t1=  0.109162 t2=  0.100610 t3=  0.209772 int= 2.047344735296221e-02
fnbik09@sr210:~/mpc/task5$
```

Выводы:

1) Во всех запусках на корневом процессе получено int ≈ 9.9999975e-01, что соответствует ожидаемому значению 1.0 с учётом дискретизации сетки 1000^3. MPI_Reduce корректно суммирует частичные результаты без потерь.
2) Время вычислений t1 стабильно уменьшается при росте числа процессов: 1.96с (1 проц.) → ~0.32с (8 проц.) → ~0.11с (27 проц.), что подтверждает равномерное распределение нагрузки по трёхмерным доменам.
3) Время обмена t2 на корневом процессе растёт с увеличением np (от 0.00003с при 1 процессе до ~0.004–0.1с при 27), что соответствует логарифмической сложности древовидной редукции в MPI.